

ADVANCED LUDO GAME ARCHITECTURE — MISSING SECTIONS DOCUMENT

This document contains all advanced production-level sections missing from the main Ludo Game documentation.

These sections are important for:

- Scalability
 - Realtime stability
 - Wallet safety
 - Production deployment
 - Security
 - Recovery systems
 - Anti-cheat protection
 - Industry-grade architecture
-

1. COMPLETE LUDO RULE ENGINE

Core Rules

Token Unlock Rule

- Token can only leave home when dice value = 6
 - If player does not get 6:
 - Turn changes
-

Consecutive 6 Rule

1st 6 → extra turn
2nd 6 → extra turn
3rd consecutive 6 → entire turn cancelled

Kill Rule

- If token lands on enemy token:
 - Enemy token returns home
 - Current player gets bonus turn
-

Safe Zone Rule

Safe zones cannot be killed.

Example safe cells:

```
[1, 9, 14, 22, 27, 35, 40, 48]
```

Home Entry Rule

- Exact dice value required
- Example:

```
Position = 54  
Need = 2  
Dice = 3  
→ Move invalid
```

Winner Rule

Player wins when:

```
All 4 tokens reach home
```

Turn Timeout Rule

```
If player inactive for 20 sec  
→ Auto skip turn
```

2. COMPLETE BOARD STATE STRUCTURE

Realtime Board State Example

```
{  
  gameId: "GAME123",
```

```
currentTurn: "player1",

diceValue: 6,

players: {
  player1: {
    color: "red",
    tokens: [
      {
        tokenId: "red_1",
        position: 12,
        isHome: false,
        completed: false
      }
    ]
  },

  player2: {
    color: "green",
    tokens: []
  }
}
```

3. MATCHMAKING QUEUE SYSTEM

Matchmaking Flow

```
Player selects bet amount
→ Join queue
→ Find same bet players
→ Wait 10 sec
→ If no player found
→ Bot joins
```

Queue Data Structure

```
{
  userId,
  betAmount,
  joinedAt,
```

```
    status
  }
```

Queue Features

- Queue timeout
 - Queue cancellation
 - Same amount matchmaking
 - Fast bot fallback
-

4. DISCONNECT & RECONNECT SYSTEM

Disconnect Flow

```
Player disconnects
→ Timer starts (30 sec)
→ Wait for reconnect
→ If reconnect success:
    continue game
→ Else:
    bot takeover OR auto lose
```

Reconnect API

Endpoint

```
POST /game/reconnect
```

Reconnect Socket Event

```
socket.emit("reconnect_game")
```

5. GAME RECOVERY SYSTEM

Purpose

If app crashes:

```
User opens app again
→ Backend checks active game
→ Restore board state
→ Resume gameplay
```

Recovery API

Endpoint

```
GET /game/active
```

Recovery Response

```
{
  "success": true,
  "game": {}
}
```

6. DETAILED MATCH HISTORY STRUCTURE

Match History Schema

```
{
  gameId,
  player1,
  player2,
  winner,
  loser,
  duration,
  totalMoves,
```

```
gameType,  
betAmount,  
coinsWon,  
createdAt  
}
```

Match History Features

- Recent matches
- Win/loss analysis
- Game duration
- Coins earned

7. PUSH NOTIFICATION SYSTEM

Notification Types

- Match found
- Opponent joined
- Wallet credited
- Daily reward available
- Tournament started

Technology

Use:

```
Firebase Cloud Messaging (FCM)
```

Example Payload

```
{  
  "title": "Match Found",  
  "body": "Your opponent is ready"  
}
```

8. WALLET TRANSACTION LOCKING

Problem

Prevent:

- Double spending
 - Duplicate reward
 - Simultaneous wallet updates
-

Solution

Use MongoDB transactions.

Example

```
const session = await mongoose.startSession()

session.startTransaction()
```

Wallet Flow

```
Start transaction
→ Deduct coins
→ Create game
→ Commit transaction
```

9. ANALYTICS & LOGGING SYSTEM

Logging Features

- API errors
 - Socket disconnects
 - Wallet failures
 - Match duration
 - Gameplay analytics
-

Recommended Tools

Error Tracking

Sentry

Analytics

Firebase Analytics

Metrics to Track

- Daily active users
- Average match duration
- Most selected bet amount
- Bot win percentage



10. ABUSE & CHAT FILTERING

Spam Protection

Rules:

Max 1 chat per 2 sec

Offensive Messages

Since quick chat is predefined:

- No custom messages allowed
 - Prevents abuse automatically
-

11. RESPONSIVE UI GUIDELINES

Device Support

- Android phones
 - Tablets
 - Different resolutions
-

UI Rules

- Use flex layouts
 - Avoid fixed widths
 - Responsive board scaling
 - Smooth animations
-

FPS Optimization

- Avoid unnecessary re-renders
 - Memoize heavy components
 - Use optimized animations
-

12. DESIGN SYSTEM

Colors

```
export const COLORS = {  
  primary: "#4F46E5",  
  background: "#FFFFFF",  
  success: "#22C55E",  
  danger: "#EF4444"  
}
```

Typography

```
heading: 24  
subHeading: 18  
body: 14
```

Button Variants

- Primary
 - Secondary
 - Outline
 - Disabled
-

13. SOCKET SCALING ARCHITECTURE

Problem

Single server socket system fails under heavy load.

Solution

Use:

Socket.io Redis Adapter

Architecture

```
Server 1
  ↓
Redis Pub/Sub
  ↓
Server 2
```

Benefits

- Shared rooms
 - Shared events
 - Multi-server realtime sync
-

14. COMPLETE TEST CASES

Wallet Tests

- Negative balance prevention
 - Duplicate reward prevention
 - Simultaneous deduction prevention
-

Gameplay Tests

- Invalid move rejection
 - Wrong turn rejection
 - Safe zone protection
 - Kill logic validation
-

Socket Tests

- Reconnect handling
 - Room joining
 - Duplicate events
-

UI Tests

- Navigation testing
 - Responsive testing
 - Animation testing
-

15. RATE LIMITING & SECURITY

Security Middleware

Install:

```
npm i helmet express-rate-limit
```

Features

- API protection
 - Rate limiting
 - Header security
 - DDOS protection basics
-

JWT Expiry

Token expires in 7 days

Rate Limit Example

100 requests / 15 min

16. ADMIN PANEL SYSTEM

Admin Features

- View users
 - View matches
 - Ban users
 - Wallet adjustments
 - Analytics dashboard
 - Match history
-

Admin APIs

Get All Users

GET /admin/users

Ban User

```
POST /admin/ban-user
```

17. CI/CD PIPELINE

Deployment Flow

```
GitHub Push  
→ Run Tests  
→ Build Project  
→ Deploy Server  
→ Restart Services
```

Recommended Tools

- GitHub Actions
- Docker
- PM2

PM2 Usage

```
pm2 start server.js
```

18. BOT DIFFICULTY LEVELS

Easy Bot

- Random moves

Medium Bot

- Prioritize kills
- Avoid danger

Hard Bot

- Strategic movement
 - Advanced predictions
 - Risk calculation
-

19. NETWORK FAILURE HANDLING

Problem

Weak internet during gameplay.

Solution

```
Auto reconnect socket
→ Sync latest board
→ Resume gameplay
```

Retry Strategy

```
Retry after:
1 sec
2 sec
5 sec
10 sec
```

20. API RESPONSE STANDARDIZATION

Success Response

```
{
  "success": true,
  "message": "Game created successfully",
}
```

```
"data": {}  
}
```

Error Response

```
{  
  "success": false,  
  "message": "Invalid move",  
  "error": {}  
}
```

FINAL PRODUCTION RECOMMENDATIONS

Use Clean Architecture

Recommended layers:

```
Routes  
→ Controllers  
→ Services  
→ Repositories  
→ Database
```

Use Constants Everywhere

Never hardcode:

- Socket events
- Chat messages
- Colors
- Game rules
- Wallet values

Use Environment Variables

Store:

- Secrets

- URLs
- API keys
- Firebase configs

inside `.env`

Production Hosting Recommendation

Backend

- AWS EC2
 - Render
 - Railway
-

Database

- MongoDB Atlas
-

Realtime Scaling

- Redis
 - Socket.io adapter
-

FINAL CONCLUSION

These advanced sections transform the Ludo Game architecture into:

- Production-ready system
- Realtime scalable application
- Wallet-safe gaming platform
- Anti-cheat architecture
- Industry-level multiplayer backend

This architecture is capable of supporting:

- Thousands of concurrent users
- Realtime multiplayer gameplay
- Competitive gaming systems
- Tournament ecosystems
- Cash-based gaming flows